the output. If `collision()` wasn't defined in the namespace we were in, PHP would next check the root namespace, which we can see happening here:

```
namespace Other\New {
    collision();
}

Output:
Hello from root namespace.
```

There is no `collision()` defined in the sub-namespace `Other\New` so we get the root version, but what if we wanted to call the root version but we were currently in another namespace?

Qualified Namespace Function Calls

```
namespace Other {
    collision();
    \collision();
}

Output:
Hello from Other namespace.
Hello from root namespace.
```

This code is executing in the `Other` namespace. If we want to call the root version of `collision()`, we have to add some extra information in our expression. The backslash character turns our second call to `collision()` into a *qualified* function name. Rather than letting PHP follow its searching rules (check current namespace, then fall back to root), we have instead explicitly asked for the root namespaced function. If we want to call `collision()` from `Other\Sub` while not in the `Other\Sub` namespace, we can do it in two ways: qualified or fully qualified.

```
namespace Other {
    Sub\collision(); // qualified - namespace path is relative to current
    \Other\Sub\collision(); // fully qualified - namespace path starts
    from root
}
```